B25 Attention and transformers from scratch

Tokens and embeddings, sequence models, attention, masks, multi-head attention, positions and the transformer block.

B25-L1 Tokens, vocabularies and embeddings

Build the vocabulary on training text with reserved IDs for padding and unknown words, encode to fixed-length integer IDs, and save the vocabulary with the model.

```
vocab = {"<pad>": 0, "<unk>": 1, "price": 2, "too": 3, "high": 4, "staff": 5,
  "great": 6}
encode("Price too high", vocab, length=5)      # [2, 3, 4, 0, 0]
encode("Staff were great", vocab, length=5)    # [5, 1, 6, 0, 0]
```

Reserved IDs in a vocabulary	0 for <pad>, 1 for <unk>. Words start at 2. Build it on training text and save it with the model.
What nn.Embedding(vocab_size, dim) does	Looks up row i for ID i: (N, L) integer IDs become (N, L, dim). Same as a one-hot vector times the table.
padding_idx=0	Row 0 starts at zero and gets no gradient, so padding never learns a meaning.
Two embedding errors	Float IDs raise a dtype error. An ID at or beyond the vocabulary size raises 'index out of range in self'.

B25-L2 Sequence models, recurrence and its limits

A recurrent network carries a hidden state step by step. Set batch_first=True, and remember its two limits: fading long-range signal and no parallelism.

```
rnn = nn.RNN(input_size=4, hidden_size=8, batch_first=True)
outputs, h_last = rnn(torch.zeros(2, 5, 4))    # (2, 5, 8) and (1, 2, 8)
```

The RNN step	h_t = tanh(x_t @ W_ih.T + h_prev @ W_hh.T + b), with the same weights at every step.
nn.LSTM outputs with batch_first=True	outputs (batch, length, hidden), plus h and c of shape (layers, batch, hidden).
batch_first default	False for RNN and LSTM: (length, batch, features). Wrong layouts often run without error. Set it and assert shapes.
Two limits of recurrent models	Gradients fade or explode over long sequences (0.9 ** 50 = 0.0052), and steps cannot run in parallel.

B25-L3 Scaled dot-product attention

Scores are Q @ K.T / sqrt(d), softmax runs across keys with the row maximum subtracted, and the output is the weighted average of the values.

```
scores = Q @ K.T / np.sqrt(d)              # (length, length)
weights = softmax(scores, axis=-1)         # each row sums to 1
output = weights @ V                       # (length, d)
```

Scaled dot-product attention	weights = softmax(Q @ K.T / sqrt(d)) across keys; output = weights @ V.
Why scale by sqrt(d)	Dot products grow like sqrt(d): about 2, 8 and 22 at d = 4, 64, 512. Scaling keeps softmax out of saturation.
Stable softmax	Subtract the maximum along the axis with keepdims=True before exp, then divide by the sum along the same axis.

Attention in PyTorch 2.15	torch.nn.functional.scaled_dot_product_attention(q, k, v) on (..., length, d) tensors.

B25-L4 Masks for padding and for the future

Mask padding and the future by setting their scores to minus infinity. Know which convention each PyTorch function uses, and test that later tokens and padding cannot change earlier outputs.

```
scores = np.where(allowed, scores, -np.inf)    # allowed: True where attention may look
```

Masking in attention	Set masked scores to minus infinity before softmax, so their weights are exactly 0.
Causal mask	np.tril(np.ones((L, L), dtype=bool)): position i sees 0 to i. F.scaled_dot_product_attention(..., is_causal=True) builds it.
PyTorch 2.15 mask conventions	scaled_dot_product_attention attn_mask: True = take part. MultiheadAttention key_padding_mask: True = ignore.
Mask tests	Changing a later token, or adding padding, must not change earlier outputs or outputs at real positions.

B25-L5 Multi-head attention and position information

Multi-head attention splits d into heads, attends in each and merges them. Attention ignores order, so add learned or sinusoidal positions to the embeddings.

Multi-head attention steps	Project to Q, K, V; split d into h heads of d // h; attend per head; merge and project with Linear(d, d).
nn.MultiheadAttention parameters	4 * d * d + 4 * d, whatever the number of heads. d must divide by num_heads.
Why positions	Attention ignores order: shuffled inputs give the same outputs, shuffled. Add learned or sinusoidal positions to the embeddings.
Sinusoidal positions	PE[pos, 2i] = sin(pos / 10000 ** (2i / d)), PE[pos, 2i + 1] = cos of the same angle.

B25-L6 The transformer block, residuals, layer norm and feed-forward

A block is attention then feed-forward, each with layer norm and a residual. Set batch_first, norm_first and dim_feedforward yourself, and pool over real tokens only.

```
x = x + attention(norm_1(x))        # tokens exchange information
x = x + feed_forward(norm_2(x))     # each token is transformed on its own
```

Pre-norm transformer block	x = x + attention(norm_1(x)); x = x + feed_forward(norm_2(x)).
Layer norm	Per token: subtract the mean, divide by sqrt(population variance + eps), then a learned scale and shift.
TransformerEncoderLayer defaults in PyTorch 2.15	batch_first=False, norm_first=False, dim_feedforward=2048, dropout=0.1. Set the first three yourself.
Pooling for classification	Mean over real positions only: sum with a 0/1 mask, divide by the count of real tokens.